Welcome to CSC331 Fall 2025 Data Structures

Professor Kevin Byron (kbyron@bmcc.cuny.edu)
Department of CIS, Room F-1030-I
Office hours: Mon 2pm-5pm (in-person and Zoom)
Office phone: 212-220-1490
Borough of Manhattan Community College/CUNY

CSC331 Fall 2025
Data Structures

Week 6:

Heap processing with user defined class
Program 4 specifications

Professor Kevin Byron
Department of CIS
Borough of Manhattan Community College/CUNY

Reminders

- Program 2 due date
  - 1159pm Fri 10-10-2025
  - late submission receives no credit
- No class Mon 10-13-2025, Mon 10-20-2025
- Follow Monday schedule on: Tue 10-14-2025, Fri 10-24-2025
- Program 3 due date
  - 1159pm Fri 10-31-2025
  - late submission receives no credit
- Exam 2
  - Section 501 Tue 11-11-2025 – Zoom 630pm-730pm
  - Section 172 Wed 11-12-2025 – In person room M-1001 630pm-730pm
    - Closed book, closed notes, no devices
  - Section 500 Wed 11-12-2025 – In person room N-453 1010am-1110am
    - Closed book, closed notes, no devices
- Program 4 due date
  - 1159pm Fri 11-21-2025
  - late submission receives no credit
- Posted on Brightspace
  - Week 6 lecture slides, C++ program with user-define heap class (chap1f.cpp), Program 4 specifications, heap331_f25.h header file

String data vs numeric data

- Find the smallest value in each list below
- Build a min-heap from each list below by pushing elements in the order given

List of strings	List of numbers
"albert"	09
"Albert"	3.14
"2-77-albert"	77
"09"	2
"albert-77"	-4
"3.14"	9

"3.14" > "2-77-albert"

"Albert" < "albert"

"A" = 65
"a" = 97
"3" = 51

"3.14" < "albert-77"

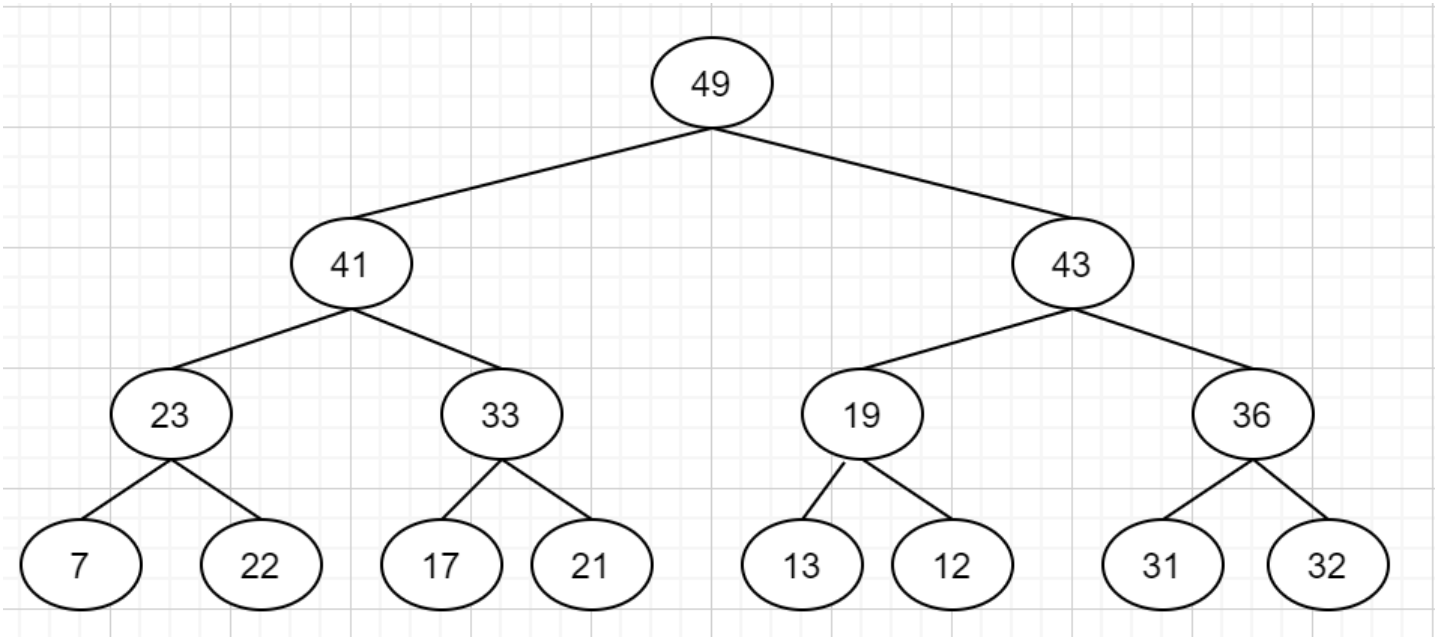
String data vs numeric data

- ASCII (American standard code for information interchange) conversion chart

Decimal - Binary - Octal - Hex – ASCII
Conversion Chart

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
0	00000000	000	00	NUL	32	00100000	040	20	SP	64	01000000	100	40	@	96	01100000	140	60	`
1	00000001	001	01	SOH	33	00100001	041	21	!	65	01000001	101	41	A	97	01100001	141	61	a
2	00000010	002	02	STX	34	00100010	042	22	“	66	01000010	102	42	B	98	01100010	142	62	b
3	00000011	003	03	ETX	35	00100011	043	23	#	67	01000011	103	43	C	99	01100011	143	63	c
4	00000100	004	04	EOT	36	00100100	044	24	$	68	01000100	104	44	D	100	01100100	144	64	d
5	00000101	005	05	ENQ	37	00100101	045	25	%	69	01000101	105	45	E	101	01100101	145	65	e
6	00000110	006	06	ACK	38	00100110	046	26	&	70	01000110	106	46	F	102	01100110	146	66	f
7	00000111	007	07	BEL	39	00100111	047	27	‘	71	01000111	107	47	G	103	01100111	147	67	g
8	00001000	010	08	BS	40	00101000	050	28	(	72	01001000	110	48	H	104	01101000	150	68	h
9	00001001	011	09	HT	41	00101001	051	29	)	73	01001001	111	49	I	105	01101001	151	69	i
10	00001010	012	0A	LF	42	00101010	052	2A	*	74	01001010	112	4A	J	106	01101010	152	6A	j
11	00001011	013	0B	VT	43	00101011	053	2B	+	75	01001011	113	4B	K	107	01101011	153	6B	k
12	00001100	014	0C	FF	44	00101100	054	2C	,	76	01001100	114	4C	L	108	01101100	154	6C	l
13	00001101	015	0D	CR	45	00101101	055	2D	-	77	01001101	115	4D	M	109	01101101	155	6D	m
14	00001110	016	0E	SO	46	00101110	056	2E	.	78	01001110	116	4E	N	110	01101110	156	6E	n
15	00001111	017	0F	SI	47	00101111	057	2F	/	79	01001111	117	4F	O	111	01101111	157	6F	o
16	00010000	020	10	DLE	48	00110000	060	30	0	80	01010000	120	50	P	112	01110000	160	70	p
17	00010001	021	11	DC1	49	00110001	061	31	1	81	01010001	121	51	Q	113	01110001	161	71	q
18	00010010	022	12	DC2	50	00110010	062	32	2	82	01010010	122	52	R	114	01110010	162	72	r
19	00010011	023	13	DC3	51	00110011	063	33	3	83	01010011	123	53	S	115	01110011	163	73	s
20	00010100	024	14	DC4	52	00110100	064	34	4	84	01010100	124	54	T	116	01110100	164	74	t
21	00010101	025	15	NAK	53	00110101	065	35	5	85	01010101	125	55	U	117	01110101	165	75	u
22	00010110	026	16	SYN	54	00110110	066	36	6	86	01010110	126	56	V	118	01110110	166	76	v
23	00010111	027	17	ETB	55	00110111	067	37	7	87	01010111	127	57	W	119	01110111	167	77	w
24	00011000	030	18	CAN	56	00111000	070	38	8	88	01011000	130	58	X	120	01111000	170	78	x
25	00011001	031	19	EM	57	00111001	071	39	9	89	01011001	131	59	Y	121	01111001	171	79	y
26	00011010	032	1A	SUB	58	00111010	072	3A	:	90	01011010	132	5A	Z	122	01111010	172	7A	z
27	00011011	033	1B	ESC	59	00111011	073	3B	;	91	01011011	133	5B	[	123	01111011	173	7B	{
28	00011100	034	1C	FS	60	00111100	074	3C	<	92	01011100	134	5C	\	124	01111100	174	7C	|
29	00011101	035	1D	GS	61	00111101	075	3D	=	93	01011101	135	5D	]	125	01111101	175	7D	}
30	00011110	036	1E	RS	62	00111110	076	3E	>	94	01011110	136	5E	^	126	01111110	176	7E	~
31	00011111	037	1F	US	63	00111111	077	3F	?	95	01011111	137	5F	_	127	01111111	177	7F	DEL

Integer heap sample



Maxheap:

49	41	43	23	33	19	36	7	22	17	21	13	12	31	32
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Heaps

- Heap processing with user defined class

```
// chap1f.cpp
// kbyron@bmcc.cuny.edu
// 3/22/19
// C program to demonstrate user-defined heap class
#include <cmath>
#include <iostream>
using namespace std;
class heapType{
  public:
    bool empty();
    int front();
    void pop();
    void push(int num);
    heapType();
  private:
    int C1I;   // child 1 index
    int C2I;   // child 2 index
    int CI;    // child index
    int done;  // status flag
    int EOL;   // end of line
    int count; // elements in the heap
    int heap[10];
    int PI;    // parent index
    void swap(int I1,int I2); // exchange heap contents
};
```

Heaps

- Heap processing with user defined class

```
int main(){
  cout<<"we will use queue functions with a heap."<<endl;
  heapType iHeap;
  iHeap.pop();
  cout << "pushing: 51 42 63 " << endl;
  iHeap.push(51);
  iHeap.push(42);
  iHeap.push(63);
  cout<<iHeap.front()<<endl;
  iHeap.pop();
  cout << "pushing: ";
  for(int i=1;i<12;i++){
    cout << (i*30)%100 << " ";
    iHeap.push((i*30)%100);
  }
  cout << endl;
  cout << "fronting and popping: ";
  while(!iHeap.empty()){
    cout << iHeap.front() << " ";
    iHeap.pop();
  }
  cout<<endl;
  cout<< "done."<<endl;
  return 0;
}
```

Heaps

- Heap processing with user defined class - pop

```
void heapType::pop(){
  if(count == 0)
    cout << "pop() failed ... heap is empty.\n";
  else{
    if(count == 1){
      count=0; EOL=0; heap[0]=0;
    }
    else{
      if(count == 2){
        count=1; EOL=1; heap[0]=heap[1]; heap[1]=0;
      }
      else{
        count--; EOL--; heap[0]=heap[EOL];
        heap[EOL]=0; done=0; PI=0; C1I=1; C2I=2;
        while (!done){
          if(C2I >= EOL){
            if(heap[PI] > heap[C1I])
              swap(PI,C1I);
            done=1;
          }
          else{
            if((heap[PI]<=heap[C1I])&&(heap[PI]<=heap[C2I]))
              done=1;
            else{
              if(heap[C1I]<heap[C2I]){
                swap(PI,C1I);
                if((C1I*2+1)>=EOL)
                  done=1;
                else
                  PI=C1I;
              }
              else{
                swap(PI,C2I);
                if((C2I*2+1)>=EOL)
                  done=1;
                else
                  PI=C2I;
              }
              C1I=PI*2+1; C2I=PI*2+2;
            }
          }
        } // while loop
      } // if count==2
    } // if count==1
  } // if count==0
}
```

Heaps

- Heap processing with user defined class - pop

```
void heapType::pop(){
  if(count == 0)
    cout << "pop() failed ... heap is empty.\n";
  else{
    if(count == 1){
      count=0; EOL=0; heap[0]=0;
    }
    else{
      if(count == 2){
        count=1; EOL=1; heap[0]=heap[1]; heap[1]=0;
      }
      else{
```

Heaps

- Heap processing with user defined class - pop

```
count--; EOL--; heap[0]=heap[EOL];
heap[EOL]=0; done=0; PI=0; C1I=1; C2I=2;
while (!done){
  if(C2I >= EOL){
    if(heap[PI] > heap[C1I])
      swap(PI,C1I);
    done=1;
  }
  else{
    if((heap[PI]<=heap[C1I])&&(heap[PI]<=heap[C2I]))
      done=1;
    else{
      if(heap[C1I]<heap[C2I]){
        swap(PI,C1I);
        if((C1I*2+1)>=EOL)
          done=1;
        else
          PI=C1I;
      }
      else{
        swap(PI,C2I);
        if((C2I*2+1)>=EOL)
          done=1;
        else
          PI=C2I;
      }
      C1I=PI*2+1; C2I=PI*2+2;
    }
  }
} // while loop
```

Heaps

- Heap processing with user defined class

```
void heapType::push(int num){
  if(count == 10){
    cout << "push(" << num << ") failed ... heap is full." << endl;
  }
  else{
    count++;
    heap[EOL]=num;
    CI=EOL;
    EOL++;
    done=0;
    while(!done){
      if(CI==0)
        done=1;
      else{
        PI=(CI-1)/2;
        if(heap[PI]<=heap[CI])
          done=1;
        else{
          swap(PI,CI);
          CI=PI;
        }
      } // if CI==0
    } // while
  } // if count==10
}
```

Heaps

- Heap processing with user defined class

```
bool heapType::empty(){
  return(count == 0);
}
// ------------------------------------------------------
void heapType::swap(int I1,int I2){ // exchange heap contents
  int T=heap[I1];
  heap[I1]=heap[I2];
  heap[I2]=T;
}
// ------------------------------------------------------

int heapType::front(){
  if(count == 0)
    cout << "front() failed ... heap is empty.\n";
  else
    return heap[0];
}
// ------------------------------------------------------
heapType::heapType(){ //default constructor
  EOL=0; count=0;
  for(int i=0; i<10; i++)
    heap[i]=0;
}
```

Queues drill problems

- D1: Given the heap below, what are the contents in location [6] after 1 pop operation?

Maxheap:

49	41	43	23	33	19	36	7	22	17	21	13	12	31	32
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Queues drill problems

- D2: Given the heap below, what are the contents in location [6] after 1 pop operation?

49	43	41	23	21	19	36	7	22	17	20	13	12	31	32
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Queues drill problems

- D3: Given the heap below, what are the contents in location [7] after the push(21) operation?

Maxheap:

49	41	43	23	33	19	36	7	22	17	21	13	12	31	32
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Queues drill problems

- D4: Given the heap below, what are the contents in location [0] after the push(21) operation?

Maxheap:

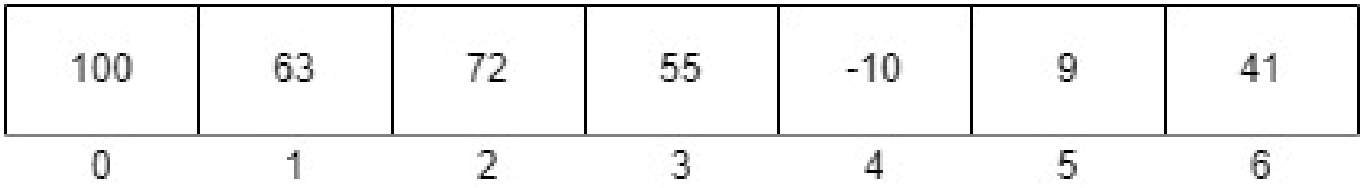
49	41	43	23	33	19	36	7	22	17	21	13	12	31	32
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Queues drill problems

a. D5: How many layers are there in a binary tree representing a heap with 127 nodes?
b. D6: How many layers are there in a binary tree representing a heap with 15 nodes?
c. D7: How many layers are there in a binary tree representing a heap with 50 nodes?

Queues drill problems

- D8: (true or false) The array contents below represent a heap.



Queues drill problems

- D9: (true or false) The array contents below represent a heap.

'623'	'Paco'	'Chan'	'bill'	'Chan'	'harry'	'79'
0	1	2	3	4	5	6

Queues drill problems

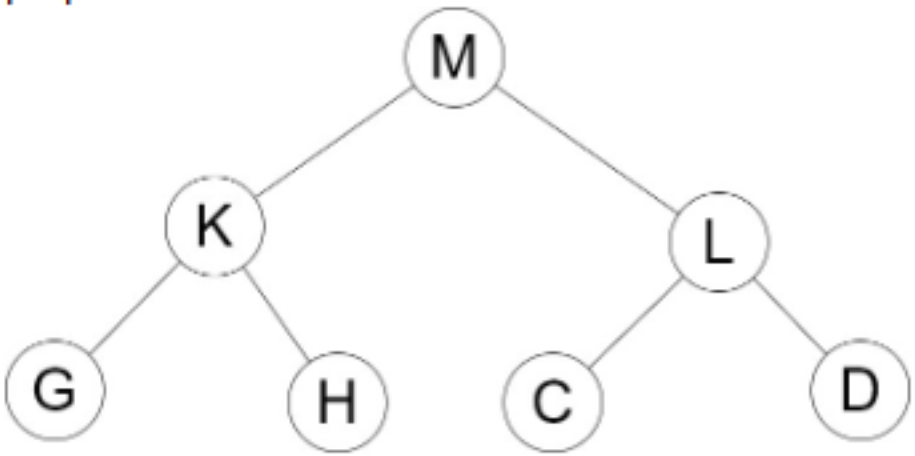
- D10: C++ coding …
  - Part of the class definition of a C++ linked-list implementation of a simple first-in-first-out queue is shown below. Show the C++ code for the front() function of the class. If the queue is empty, return the value -999.

```
class queueType{
  public:
    bool empty();
    void pop();
    void push(int num);
    int front();
    queueType();
  private:
    struct nodeType{
      int info;
      nodeType *link;
    };
  nodeType *head,*rear,*temp;
};
```

Queues drill problems

- D11: Heap pop

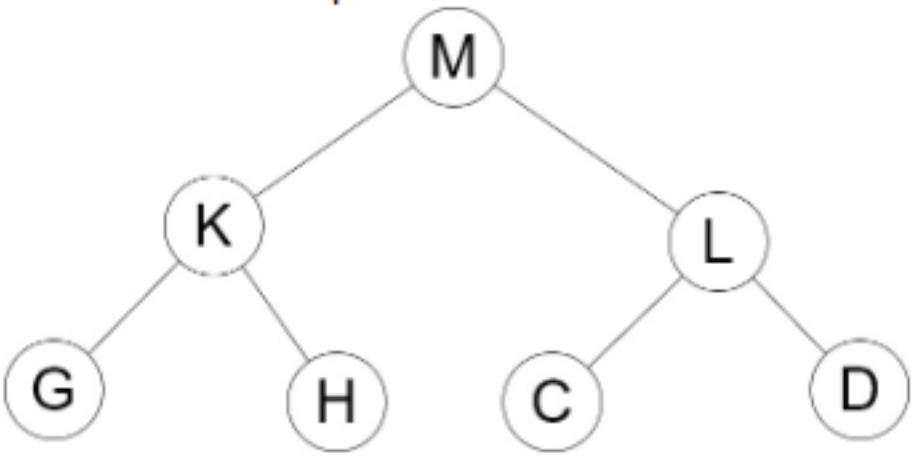
Given the heap below, identify the parent of the L node after one pop.



Queues drill problems

- D12: Heap push

Given the heap below, identify the parent of the L node after pushing P onto the heap.



Queues drill problems

- D13: Show the output from the following …

```
cout<<"queue operations ... "<<endl;
queue<float> myQue;
myQue.push(7);
myQue.pop();
myQue.push(8);
myQue.pop();
myQue.push(9);
cout<<myQue.front()<<endl;
for(int i=1;i<12;i+=5)
  myQue.push(i);
while(!myQue.empty()){
  cout << myQue.front() << " ";
  myQue.pop();
}
cout<<endl;
cout<< "done."<<endl;
```

Queues drill problems

- D14: Show the output from the following …

```
maxHeap myHeap;
myHeap.push(9);
myHeap.pop();
myHeap.push(7);
myHeap.push(8);
cout<<myHeap.front()<<endl;
for(int i=1;i<12;i+=5)
  myHeap.push(i);
while(!myHeap.empty()){
  cout << myHeap.front() << endl;
  myHeap.pop();
}
cout<< "done."<<endl;
```

Queues drill problems

- Drill problem solutions …

```
D1. 32
D2. 36
D3. 21
D4. 49
D5. 7
D6. 4
D7. 6
D8. T
D9. F
D10. int queueType::front() {
       if (head == NULL)
         return -999;
       return head->info;
     }
D11. no parent
D12. P
D13. 9
     9 1 6 11
     done.
D14. 11
     8
     7
     6
     1
     done.
```

Assignment

- Read Malik chapter 10 – heaps
- Read Malik chapter 11 – binary trees
- Read Malik chapter 12 - graphs